Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний інститут імені Ігоря Сікорського"
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт

з лабораторної роботи № 8 з дисципліни
«Алгоритми та структури даних-1.
Основи алгоритмізації»

«Дослідження алгоритмів пошуку та сортування»

Варіант 9

Виконав студент ІП-15, Дзюбенко Даниїл Дмитрович
(шифр, прізвище, ім'я, по батькові)

Перевірив Вєчерковська Анастасія Сергіївна
(прізвище, ім'я, по батькові)

Київ 2021

Лабораторна робота 8

Дослідження алгоритмів пошуку та сортування

Мета – дослідити алгоритми пошуку та сортування, набути практичних навичок використання цих алгоритмів під час складання програмних специфікацій.

Варіант 9

Задача.

9	6 x 6	Цілий	Із додатних значень елементів головної діагоналі двовимірного масиву. Відсортувати методом Шела за зростанням.

Постановка задачі.

Спочатку нам потрібно описати двовимірний масив та ініціалізувати його випадковими значеннями. Потім нам потрібно створити одновимірний масив, який буде містити додатні значення елементів головної діагоналі двовимірного масиву. Після цього треба відсортувати одновимірний масив методом Шела за зростанням. Усі окремі дії я буду виконувати за допомогою функцій. Користувач нічого не повинен вводити.

Математична модель.

Змінна	Тип	Ім'я	Призначення
Розмірність масиву	Цілий	N	Початкове дане
1-й масив	Цілий	Arr	Початкове дане
2-й масив	Цілий	Arr1	Результат
Функція, яка створює 1-й та 2-й масиви	Процедура	createArray12	Проміжні дані
Функція, яка виводить 1-й масив	Процедура	printArray1	Проміжні дані
Функція, яка виводить 2-й масив	Процедура	printArray2	Проміжні дані
Функція, яка виводить відсортований масив	Процедура	printSortedArray	Проміжні дані
Функція, яка сортує масив методом Шела	Процедура	sortShell	Проміжні дані

Функція createArray12 за допомогою вкладеності циклів заповнює двовимірний масив розмірністю NxN випадковими числами, також за допомогою умовного оператора заповнює одновимірний масив. Функція printArray1 виводить двовимірний масив. Функція printArray2 виводить одновимірний масив. Функція printSortedArray виводить вже відсортований одновимірний масив. Функція sortShell сортує одновимірний масив методом Шела. Виклик усіх функцій відбувається в основній програмі. Для не дорівнює будемо використовувати знак «!=». Для знаходження випадкового числа в діапазоні від -100 до 100 в псевдокоді та блок-схемі я буду використовувати позначення «random(-100,100)».

Розв'язання:

Крок 1. Визначимо основні дії.

Крок 2. Деталізуємо дію створення 1-го та 2-го масивів.

Крок 3. Деталізуємо дію сортування 2-го масиву.

Крок 4. Деталізуємо дію виводу 2-го масиву.

Псевдокод.

Крок 1	Крок 2	Крок 3

Початок
Створення 1-го та 2-го масивів
Сортування 2-го масиву
Вивід 2-го масиву
Кінець
	Початок
	N = 10;
	arr[n][n], arr1[n];
	createArray12(arr, arr1, n);
	Сортування 2-го масиву
	Вивід 2-го масиву
	Кінець
	
	Підпрограма createArray12:
	Початок
	Повторити
	Для i від 0 до n
	Повторити
	Для j від 0 до n
	arr[i][j] = random(-100,100)
	Все повторити
	Якщо arr[i][j]>0
	То arr1[i] = arr[i][j]
	Інакше arr1[i] = 0
	Все якщо
	Все повторити
	Кінець
		Початок
		N = 10;
		arr[n][n], arr1[n];
		createArray12(arr, arr1, n);
		sortShell(arr1, n);
		Вивід 2-го масиву
		Кінець
		
		Підпрограма createArray12:
		Початок
		Повторити
		Для i від 0 до n
		Повторити
		Для j від 0 до n
		arr[i][j] = random(-100,100)
		Все повторити
		Якщо arr[i][j]>0
		То arr1[i] = arr[i][j]
		Інакше arr1[i] = 0
		Все якщо
		Все повторити
		Кінець
		
		Підпрограма sortShell:
		Початок
		swap = 0, comp = 0, incr = n/2;
		temp, k;
		Поки incr>0
		Повторити
		Повторити
		Для i від 0 до incr
		Повторити
		Для j від 0 до n з кроком +incr
		temp = arr[j]
		Повторити
		Для k від j-incr до k>=0 та temp<arr[k] з кроком -incr
		comp++
		swap++
		arr[k+incr] = arr[k]
		Все повторити
		arr[k+incr] = temp
		swap++
		Все повторити
		Все повторити
		comp++
		Якщо incr/2 != 0
		То incr = incr/2
		Інакше якщо incr == 1
		То incr = 0
		Інакше incr = 1
		Все якщо
		Все якщо
		Все повторити
		Кінець

Крок 4

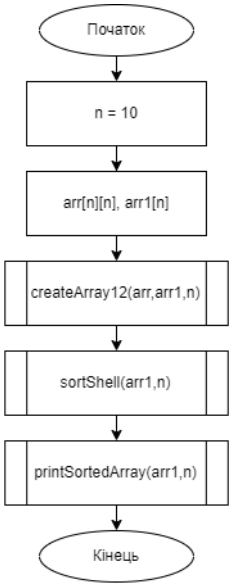
Початок
  N = 10;
  arr[n][n], arr1[n];
  createArray12(arr, arr1, n);
  sortShell(arr1, n);
  printSortedArray(arr1, n);
Кінець

Підпрограма createArray12:
Початок
  Повторити
  Для i від 0 до n
    Повторити
    Для j від 0 до n
      arr[i][j] = random(-100,100)
    Все повторити
    Якщо arr[i][j]>0
      То arr1[i] = arr[i][j]
      Інакше arr1[i] = 0
    Все якщо
  Все повторити
Кінець

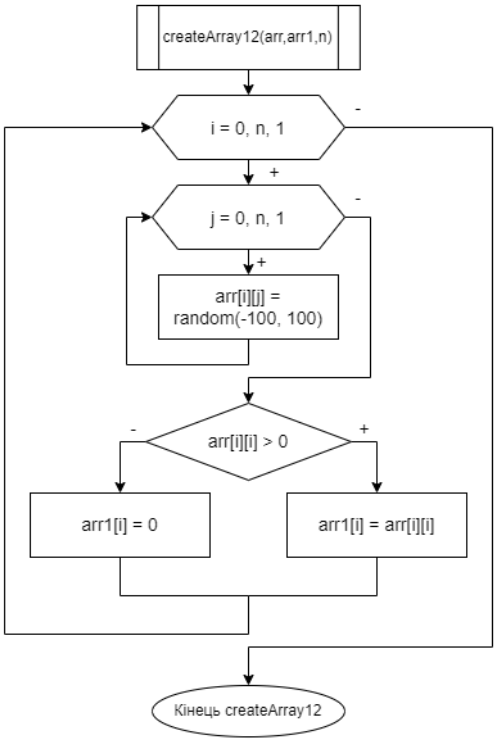
Підпрограма sortShell:
Початок
  swap = 0, comp = 0, incr = n/2;
  temp, k;
  Поки incr>0
  Повторити
    Повторити
    Для i від 0 до incr
      Повторити
      Для j від 0 до n з кроком +incr
        temp = arr[j]
        Повторити
        Для k від j-incr до k>=0 та temp<arr[k] з кроком -incr
          comp++
          swap++
          arr[k+incr] = arr[k]
        Все повторити
        arr[k+incr] = temp
        swap++
      Все повторити
    Все повторити
    comp++
    Якщо incr/2 != 0
      То incr = incr/2
      Інакше якщо incr == 1
        То incr = 0
        Інакше incr = 1
      Все якщо
    Все якщо
  Все повторити
Кінець

Підпрограма printSortedArray:
Початок
  Повторити
  Для i від 0 до n
    Якщо arr[i] != 0
      То виводити arr[i]
    Все якщо
  Все повторити
Кінець

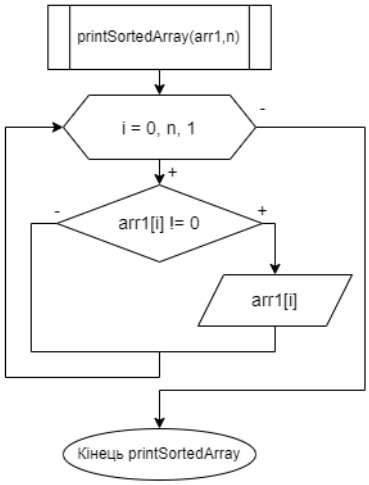
Блок-схема.



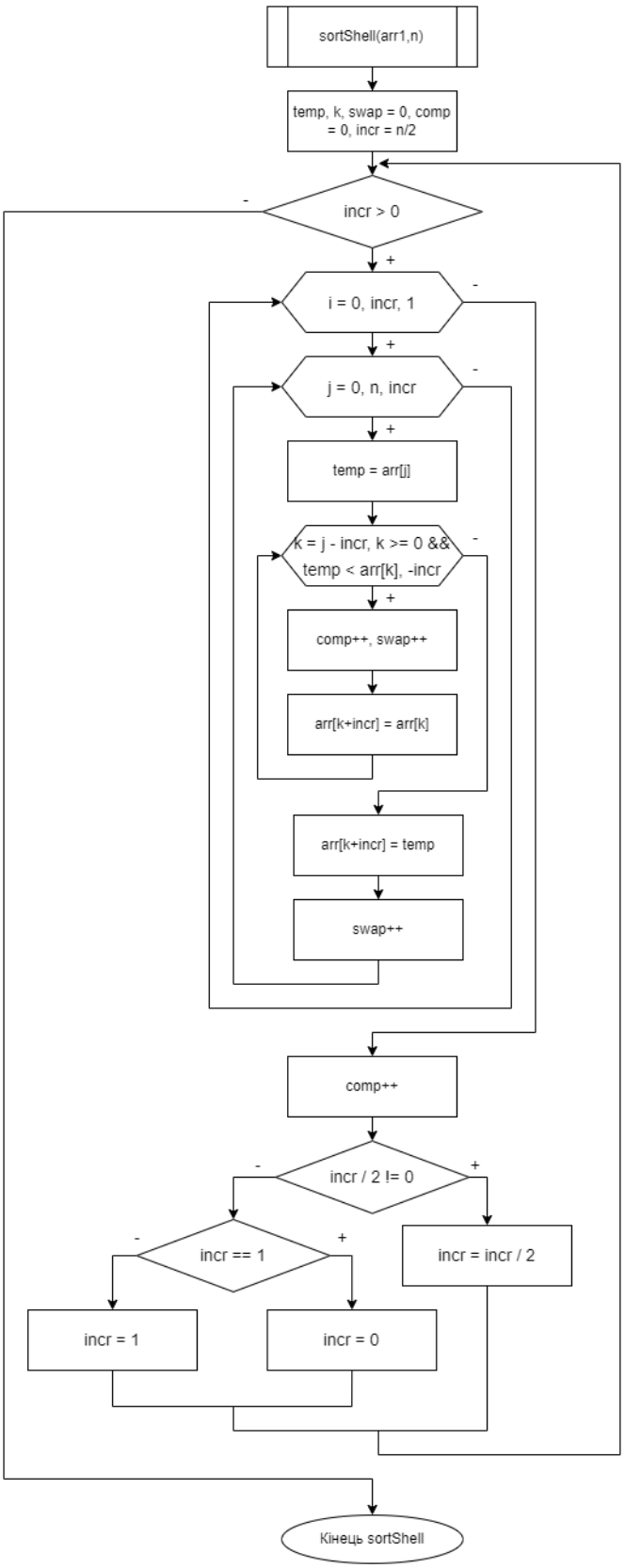
Підпрограма



Підпрограма



Підпрограма



Код програми.

```
1   #include <iostream>
2   #include <ctime>
3   #include <iomanip>
4   using namespace std;
5
6   void createArray12(int * arr[], int * arr1, int n);
7   void printArray1(int * arr[], int n);
8   void printArray2(int * arr1, int n);
9   void printSortedArray(int * arr1, int n);
10  void sortShell(int * arr, int n);
11
12  int main(){
13      srand(time(NULL));
14      int n = 6, arr[n][n], arr1[n];
15      int *p[n];
16      for (int i = 0; i < n; i++) {
17          p[i] = arr[i];
18      }
19
20      createArray12(p, arr1, n);
21      cout<<"Array 1:"<<endl;
22      printArray1(p,n);
23      cout<<endl;
24      cout<<"Array 2:"<<endl;
25      printArray2(arr1, n);
26      sortShell(arr1, n);
27      cout<<endl;
28      cout<<"\nSorted array: "<<endl;
29      printSortedArray(arr1, n);
30
31      system("pause");
32  }
33
34  void createArray12(int ** arr, int * arr1, int n){
35      for (int i = 0; i < n; i++) {
36          for (int j = 0; j < n; j++) {
37              arr[i][j] = rand()%(200)-100;
38          }
39          if (arr[i][i]>0) {
40              arr1[i] = arr[i][i];
41          } else {
42              arr1[i] = 0;
43          }
44      }
45  }
```

```
void printArray1(int * arr[], int n){
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout<<setw(5)<<arr[i][j];
        }
        cout<<endl;
    }
}

void printArray2(int * arr1, int n){
    for (int i = 0; i < n; i++) {
        cout<<setw(5)<<arr1[i];
    }
}

void printSortedArray(int * arr1, int n){
    for (int i = 0; i < n; i++) {
        if(arr1[i] != 0) {
            cout<<setw(5)<<arr1[i];
        }
    }
    cout<<endl;
}

void sortShell(int * arr, int n){
    int swap = 0, comp = 0, incr = n/2, temp, k;
    while (incr>0) {
        for (int i = 0; i < incr; i++) {
            for (int j = 0; j < n; j+=incr) {
                temp = arr[j];
                for (k = j-incr; k >= 0 && temp<arr[k]; k-=incr) {
                    comp++;
                    swap++;
                    arr[k+incr] = arr[k];
                }
                arr[k+incr] = temp;
                swap++;
            }
        }
        comp++;
        if (incr/2 != 0) {
            incr = incr/2;
        } else if (incr == 1) {
            incr = 0;
        } else{
            incr = 1;
        }
    }
}
```

Тестування програми.

```
Array 1:
    5  -86   -7   74   37  -56
   49   66   30  -95  -78   36
    3   92    5  -63   77   99
  -40  -95  -61  -12   33    9
  -87   84   91   60   46    9
  -60   53   98   22  -90  -93

Array 2:
    5   66    5    0   46    0

Sorted array:
    5    5   46   66
Для продолжения нажмите любую клавишу . . .
```

Висновки.

Сьогодні я дослідив алгоритми пошуку та сортування, набув практичних навичок використання цих алгоритмів під час складання програмних специфікацій. В результаті роботи я отримав алгоритм сортування послідовності за зростанням методом Шела. Дискретував задачу на кроки: визначив основні дії, деталізував дію створення 1-го та 2-го масивів, деталізував дію сортування 2-го масиву та деталізував дію виводу 2-го масиву.